

1. NumPy asoslari va Bir o'lchovli massivlar (1D Arrays)

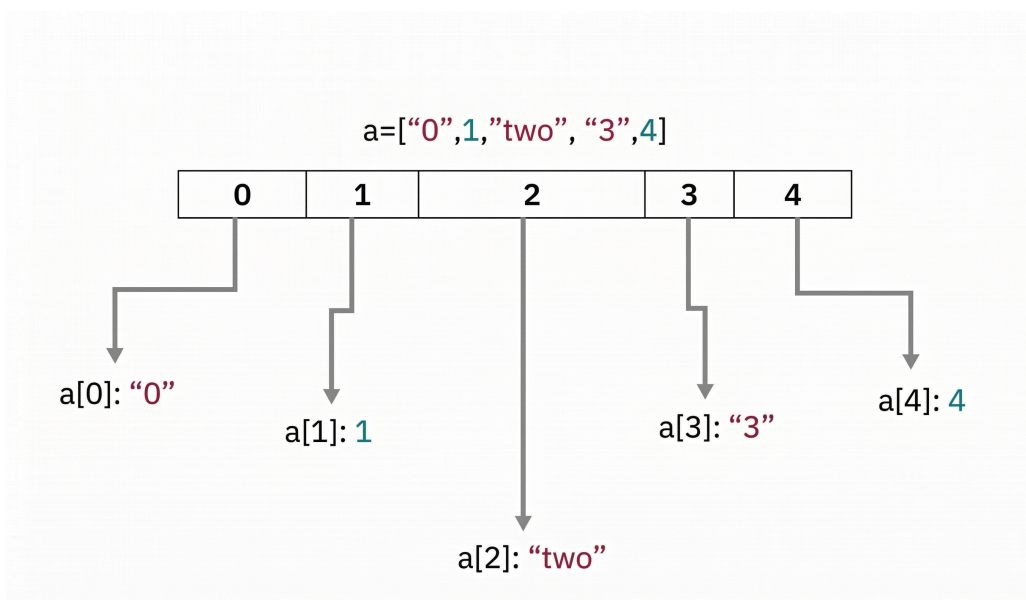
NumPy nima? NumPy — bu ma'lumotlar fani (Data Science) va ilmiy hisoblashlarda keng qo'llaniladigan kutubxona bo'lib, u juda ko'p foydali funksiyalarga ega. Uning asosiy afzalliklari oddiy Python ro'yxatlariga (lists) nisbatan ancha **yuqori tezlik** va **xotiradan tejamkorlik** hisoblanadi. Shuningdek, NumPy mashhur Pandas kutubxonasining asosi hamdir.

Reja:

1. NumPy asoslari va massiv yaratish
2. Indeksash va qirqish (Slicing)
3. Asosiy operatsiyalar (Vektor amallari)
4. Universal funksiyalar (Universal functions)

1. NumPy asoslari va Massiv yaratish

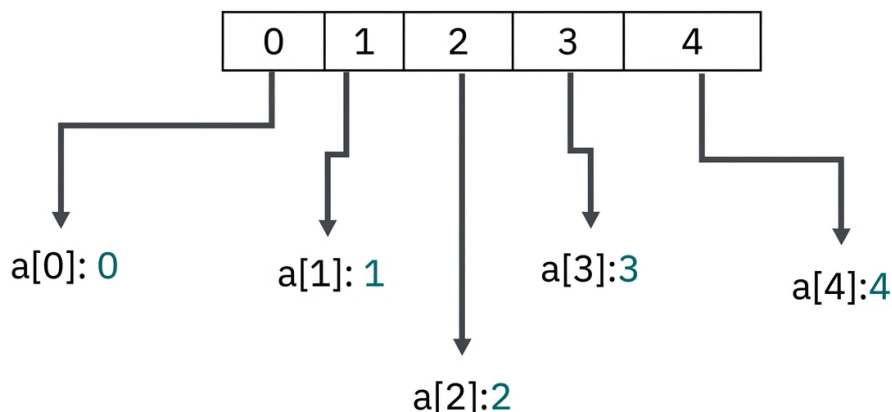
Oddiy Python ro'yxati (list) turli ma'lumotlarni saqlash imkonini beradi. Har bir element o'z indeksiga ega bo'lib, ular `0` dan boshlanadi.



1-rasm. Python ro'yxatlarida (list) elementlarning indekslanishi.

NumPy massivi (`ndarray`) ham xuddi shunga o'xshaydi, lekin u odatda belgilangan o'lchamga ega bo'ladi va uning barcha elementlari **bir xil ma'lumot turida** (masalan, faqat butun sonlar - `integer` yoki haqiqiy sonlar - `float`) bo'lishi kerak.

```
import numpy as np
a=np.array ( [0,1,2,3,4])
```



2-rasm. NumPy massivlarida elementlarning indekslanishi.

NumPy kutubxonasini ishlatish uchun avval uni chaqirib olamiz:

```
import numpy as np
```

Python ro'yxatini NumPy massiviga aylantirish (Casting):

```
# Oddiy Python ro'yxati
my_list = [0, 1, 2, 3, 4]

# Ro'yxatni NumPy massiviga aylantirish
a = np.array(my_list)
print("Massiv:", a)
```

Massiv: [0 1 2 3 4]

Massiv atributlarini tekshirish

Massiv haqida ma'lumot olish uchun bir qator atributlardan foydalanamiz:

```
# Obyekt turini tekshirish
print("Obyekt turi:", type(a)) # Natija: numpy.ndarray

# Elementlarning ma'lumot turini tekshirish
# (Masalan: 64-bitli butun son)
print("Elementlar turi:", a.dtype) # Natija: int64

# Massivdagi elementlar soni
print("Elementlar soni (size):", a.size) # Natija: 5

# Massivning o'lchovlar soni (Rank/Dimension)
print("O'lchovlar soni (ndim):", a.ndim) # Natija: 1
```

```
# Massivning shakli (Tuple ko'rinishida)
print("Massiv shakli (shape):", a.shape) # Natija: (5,)
```

```
Obyekt turi: <class 'numpy.ndarray'>
Elementlar turi: int64
Elementlar soni (size): 5
O'lchovlar soni (ndim): 1
Massiv shakli (shape): (5,)
```

Xuddi shunday, biz haqiqiy sonlardan (float) iborat massivlarni ham yaratishimiz mumkin:

```
b = np.array([3.1, 11.02, 6.2, 213.2, 5.2])
print("Elementlar turi:", b.dtype) # Natija: float64
```

```
Elementlar turi: float64
```

2. Indeksash va Qirqish (Indexing and Slicing)

Ro'yxatlardagi kabi, NumPy massivlarida ham elementlarga murojaat qilish va ularni o'zgartirish kvadrat qavslar `[]` yordamida amalga oshiriladi.

Qiymatlarni o'zgartirish:

```
c = np.array([20, 1, 2, 3, 4])

# Birinchi elementni (0-indeks) 100 ga o'zgartirish
c[0] = 100
print("O'zgartirilgan massiv:", c) # Natija: [100, 1, 2, 3, 4]

# Beshinchi elementni (4-indeks) 0 ga o'zgartirish
c[4] = 0
print("O'zgartirilgan massiv:", c) # Natija: [100, 1, 2, 3, 0]
```

```
O'zgartirilgan massiv: [100  1  2  3  4]
```

```
O'zgartirilgan massiv: [100  1  2  3  0]
```

Qirqish (Slicing):

Biz massivning ma'lum bir qismini kesib olib, yangi massivga o'zlashtirishimiz mumkin. Indeks `[boshlanish:tugash]` shaklida beriladi va "tugash" indeksi hisobga olinmaydi.

```
# 1-indeksdan boshlab 3-indeksigacha (4 kirmaydi) kesib olish
d = c[1:4]
print("Qirqilgan massiv:", d) # Natija: [1, 2, 3]

# Massivning bir nechta elementini birdaniga o'zgartirish
c[3:5] = 300, 400
```

```
print("Yangi qiymatlar bilan massiv:", c)
# Natija: [100, 1, 2, 300, 400]
```

Qirgilgan massiv: [1 2 3]

Yangi qiymatlar bilan massiv: [100 1 2 300 400]

3. Asosiy Operatsiyalar (Vektor amallari)

NumPy matematik amallarni bajarishni juda osonlashtiradi. Oddiy Python bilan sikl (`for`) yordamida ko'p qatorli kod yozish o'rniga, NumPy'da buni bir qatorda, hamda juda tez amalga oshirish mumkin. Keling, bu amallarni Evklid vektorlari misolida ko'rib chiqamiz.

Vektorlarni qo'shish va ayirish ($z = u \pm v$)

```
u = np.array([1, 0])
v = np.array([0, 1])

# Vektorlarni qo'shish
z_add = u + v
print("Qo'shish natijasi:", z_add) # Natija: [1, 1]

# Vektorlarni ayirish
z_sub = u - v
print("Ayirish natijasi:", z_sub) # Natija: [1, -1]
```

Qo'shish natijasi: [1 1]

Ayirish natijasi: [1 -1]

(Izoh: Agar buni oddiy Pythonda qilsak, har bir element uchun sikl aylantirishga to'g'ri kelardi. NumPy esa buni to'g'ridan-to'g'ri barcha elementlarga qo'llaydi, bu kattaroq o'lchamdagi datalarda o'ta yuqori unumdorlikni beradi).

Skalyarga ko'paytirish (Array multiplication with a scalar)

Vektorni biron-bir doimiy songa ko'paytirganda, uning har bir elementi shu songa ko'paytiriladi.

```
y = np.array([1, 2])
z = 2 * y
print("Skalyarga ko'paytirish:", z) # Natija: [2, 4]
```

Skalyarga ko'paytirish: [2 4]

Ikki massivning Hadamard ko'paytmasi (Element-wise product)

Bunda ikkita massivning mos indeksdagi elementlari bir-biriga ko'paytiriladi.

```
u = np.array([1, 2])
v = np.array([3, 2])

z = u * v
print("Hadamard ko'paytmasi:", z) # Natija: [3, 4] (1*3 va 2*2)
```

Hadamard ko'paytmasi: [3 4]

Nuqtali ko'paytma (Dot product)

Nuqtali ko'paytma — bu mos elementlarni ko'paytirib, so'ngra ularning yig'indisini hisoblash demakdir. Buni `np.dot()` funksiyasi yordamida bajaramiz: $u^T v$.

```
u = np.array([1, 2])
v = np.array([3, 1])

result = np.dot(u, v)
print("Nuqtali ko'paytma:", result) # Natija: 5 (1*3 + 2*1)
```

Nuqtali ko'paytma: 5

Massivga konstanta qo'shish (Broadcasting)

Agar massivga bitta butun son qo'shilsa, NumPy bu sonni avtomatik ravishda massivning **har bir elementiga** qo'shib chiqadi. Bu xususiyat **Broadcasting** deb ataladi.

```
u = np.array([1, 2, 3, -1])
z = u + 1
print("Broadcasting natijasi:", z) # Natija: [2, 3, 4, 0]
```

Broadcasting natijasi: [2 3 4 0]

4. Universal Funksiyalar va Grafik chizish

NumPy massivlari ustida ishlovchi ko'plab tayyor matematik funksiyalar mavjud.

Asosiy statistik va matematik funksiyalar

```
a = np.array([1, -1, 1, -1])

# O'rtacha qiymatni hisoblash
mean_a = a.mean()
print("O'rtacha qiymat:", mean_a) # Natija: 0.0

b = np.array([1, -2, 3, 4, 5])

# Maksimal qiymatni topish
```

```
max_b = b.max()
print("Eng katta element:", max_b) # Natija: 5

# Pi qiymatidan foydalanish
print("Pi qiymati:", np.pi)
```

O'rtacha qiymat: 0.0
Eng katta element: 5
Pi qiymati: 3.141592653589793

Massivlarga matematik funksiyalarni qo'llash (Masalan: sinus)

Biz butun bir massiv elementlariga bitta funksiyani birdaniga qo'llashimiz mumkin:

```
# Radianlarda massiv yaratamiz
x = np.array([0, np.pi/2, np.pi])

# Har bir elementning sinusini hisoblaymiz
y = np.sin(x)
print("Sinus qiymatlari:", y)
# Natija: [0.0, 1.0, 0.0] ga yaqin qiymatlar
```

Sinus qiymatlari: [0.0000000e+00 1.0000000e+00 1.2246468e-16]

linspace funksiyasi

`np.linspace(start, stop, num)` ma'lum bir oraliqda tekis taqsimlangan raqamlar ketma-ketligini yaratish uchun juda qulay funksiyadir.

```
# -2 dan 2 gacha bo'lgan oraliqda jami 5 ta nuqta yaratish
np.linspace(-2, 2, num=5)
# Natija: array([-2., -1., 0., 1., 2.])

# Xuddi shu oraliqda 9 ta nuqta yaratish
np.linspace(-2, 2, num=9)
# Natija: array([-2. , -1.5, -1. , -0.5, 0. , 0.5, 1. , 1.5, 2. ])
```

array([-2. , -1.5, -1. , -0.5, 0. , 0.5, 1. , 1.5, 2.])

5. Matematik funksiyalarni grafikda tasvirlash (Plotting)

`linspace` yordamida funksiyalar grafigini osongina chizish mumkin. Buning uchun `matplotlib` kutubxonasidan foydalanamiz. Jupyter Notebook'da grafikni to'g'ridan-to'g'ri ko'rish uchun `%matplotlib inline` sehrli (magic) buyrug'ini yozish tavsiya etiladi.

```
# Grafik chizish uchun kutubxonani chaqiramiz
import matplotlib.pyplot as plt
%matplotlib inline
```

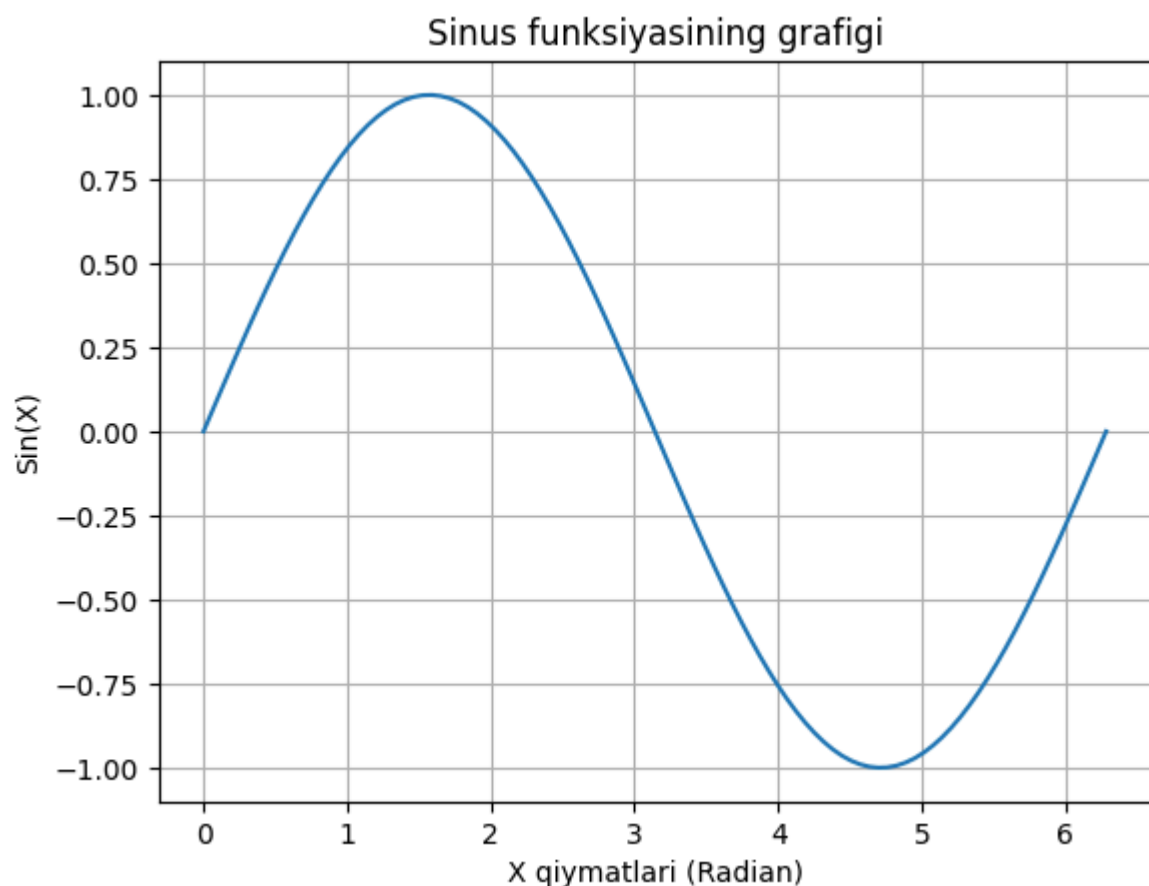
```

# 0 dan 2*Pi gacha bo'lgan oraliqda
# 100 ta nuqta yaratamiz (X o'qi uchun)
x = np.linspace(0, 2*np.pi, 100)

# Har bir X nuqtasi uchun Sinus qiymatini hisoblaymiz (Y o'qi uchun)
y = np.sin(x)

# Grafikni chizish
plt.plot(x, y)
plt.title("Sinus funksiyasining grafigi")
plt.xlabel("X qiymatlari (Radian)")
plt.ylabel("Sin(X)")
plt.grid(True)
plt.show()

```



Ushbu amallar NumPy kutubxonasining naqadar kuchli va foydali ekanligini ko'rsatadi. Bu ma'lumotlar fani, sun'iy intellekt va muhandislik hisob-kitoblarida ma'lumotlarni qayta ishlashning asosiy tayanchlaridan biri hisoblanadi.

Qo'shimcha misollar va laboratoriya ishlari uchun numpy.org saytiga murojaat qiling.